The "right" kind of context compaction for a coding agent is desperately wrong for a long-lived persona that might be engaging in multiple simultaneous conversations. And the only way to see if the credentials management tools work is to try to do something crazy, like sign into GitHub from a subagent session.

And sure, I could take this feedback into a Claude Code (or Codex) session and work it through. But I've got too many projects in flight and there's very little reason for me to be in the loop on most of this work. I would just slow it down. So I told Claude Code to use slackline to talk to "@Ada-sen" about the work it was doing.

Claude pinged Ada in #bot-testing to describe the work we'd just done and to ask Ada if it could test it out. Ada dutifully fired up a subagent and tried to log into GitHub. Claude watched the logs to see if the credential proxy did the right thing. It did not.

That kicked off a number of days of iteration. Claude would propose a change to Ada. Ada would review Claude's spec, raising questions or concerns. Claude would update the spec. Once Claude and Ada were in agreement, Claude would build an updated version of Ada, make sure Ada wasn't in the middle of something, and then deploy the update. Once Ada was up and running again, Claude would @Ada-sen, explain the test it wanted to run, and wait for results.

Once a bit of functionality works, Claude checks in with Ada to see what could make it simpler and easier to use. Ada typically kicks off some subagents to see whether they stumble through the new feature and then reports back.

At one point, I wrote to Claude "I've got to get to bed. When this project is done, check with Ada to see whatever quality of life features you could build." I woke up to a laundry list of about a dozen harness improvements. Overnight, Claude had solicited a wishlist from Ada. Ada ranked a bunch of requests. Claude ordered them and wrote out a spec for everything it was comfortable building. Ada reviewed the specs, then Claude built the features. Ada tested them out and requested changes. Claude updated the features until Ada was happy. Once Ada signed off, Claude merged the changes to main and wrote up an after-action report for me.

It's been pretty magical to watch.

FRED TALKS

15th August 2026

CONTENTS

The Self-Help Trap: What 20+ Years of “Optimizing” Has Taught Me

TIM FERRISS · 08 MAR 2026 · SOURCE

One danger of modern self-help.

> **“We cannot reason ourselves out of our basic irrationality. All we can do is to learn the art of being irrational in a reasonable way.”**
> — Aldous Huxley, *Island*

It was cold out, but none of us were cold.

I sat with five men in the mountains of Montana. As the sun set, the fire in the center cast dancing light on our faces. Reclined against fallen trees in a tight circle, we ate mushrooms and fish we’d found under trees and along streams. The whole crew burst into laughter yet again, and one of the guides passed around a fresh batch of pine needle tea.

Bathed in warmth, I took off a layer and glanced skyward through an opening in the trees. The stars shone like crystals on black velvet, and the show—the biggest meteor shower of the year—was starting.

In that moment, there was nothing to do. Nothing to improve. Nothing to fix.

It was perfect.

***

The older I get, the more I think that self-help can be a trap. Sometimes the cure is worse than the disease. I say this after ~20 years of writing self-help and a lifetime of consuming it.

Spend enough time in the world of “improvement,” and you’ll notice something strange: The people most obsessed with self-help are often the least helped by it. Behind the smiles and motivational quotes, behind closed doors and after a drink or two, the truth is that they’re not able to outsmart their worries.

On one hand, perhaps this unhappiness is precisely what lands one in self-development in the first place, right? I long assumed this about myself, and it’s partially true.

On the other hand, what if self-help *itself* is actually creating or amplifying unhappiness?

- No agent has intentional direct exposure to credentials (with one very real gap that I haven't solved yet)

All credentials for the agent live in a 1Password vault. All outgoing https traffic goes through a onecli-style transparent MITM (man in the middle) proxy. When a subagent wants to do something that requires a credential, it runs a command that asks an *arbiter* agent running in another container for permission to use that credential. If the arbiter decides the request is reasonable, it provides the subagent with a random string they can use in place of the requested credential. When the agent uses that random string in an outbound request to the right remote host, the MITM proxy rewrites the random string into the real credential on the fly.

The case that's a little more complicated is when the agent wants to do something that requires a browser, like logging into Gmail as themselves or using a service that has no API. For a bunch of reasons, the MITM rewrite trick does not work as reliably in a browser context. Often the site/app's javascript plays games with the password to hash the password before sending it. Or does some other weird validation. Or is actually sending it to a different backend service. So far, the best I've come up with keeps the credential out of the agent's transcript and prevents unintentional disclosure. In these cases, the subagent running in the container generates a random temporary password, uses superpowers-chrome to fill it into the password field, and then runs a command that asks the arbiter agent (running in its own container) for help. The arbiter, if it decides the request is reasonable, instructs a helper tool to log into the subagent's container and replace the random password with the real one. It's not perfect and there's more we can do to lock things down, but it's a better first pass than anything I've had to date.

What actually got me writing today was not this architecture, but the *development pattern* I've been using to build this. On the one side, I've got a Claude Code session. I hooked it up to our Slack instance with a self-provisioned commandline slack client Drew built called Slackline - Slackline is designed to make it easy for agents that don't *live* on Slack to participate in discussions there. It has primitives for sending messages, reading chats, waiting for replies, etc. The whole command-line slackline tool has been built for agents as its primary user. Sure, a human *could* use it to use slack from the commandline. And sure, it has enough of a json api that you could use it as part of traditional automation. But it's *for* agents.

It took a bit of iteration to get to the point where the new Sen 2.0 agentic colleague harness was live on Slack. But once it was there, I could talk through its experience of its tools and memory and credentials and workspaces. And the issues showed up pretty quickly.

And we have a small fleet of "Sen" personal assistant agents. They're **claw-esque*, but date from before openclaw was a thing. Each one has been trained to be an executive assistant, using a set of skills built on some of the resources that excellent human EAs use to learn their craft. Sen triages my mail, puts together a daily brief, does research tasks (both for me and independently), interacts with Linear, etc.

They have the standard set of affordances - They can chat on Slack. They use tools. They've got a task scheduler that can wake them up and re-inject a prompt they set for themselves at a specific time or on a schedule they define. They can use and author skills. Mine has a dropbox folder it shares with me. When we moved Sen to the cloud, I lost one of my favorite dumb features - Sen used to be able to print things directly. Sen (v1) was built on top of the Claude Agents SDK.

For the past month or so, I've been doing initial bringup of a "new" iteration of Sen, intended to be more of a colleague than an assistant. This time around, I'm building it from the ground up on top of an agents SDK I control. Lace got its start in May 2025 as my take on a command line coding agent. Over the past year, it's mutated a few times, first growing a web interface, then flipping to an ACP-derived wire protocol so you can connect any client you want to it. It speaks a bunch of different flavors of model provider API, manages caching, subagents, tools, skills, and all the other stuff you might expect. It has supported subagents in isolated containers for the better part of a year. One of the latest things to land has been the ability to run subagents locally and to project *all* of their tools into a container, such that the agent believes it is running in that container.

The thing I've been working on over the past few weeks is the credentials story for the new Sen agent running on top of Lace. Right now, I'm building with a design that assumes the agent has its own credentials for any service that it accesses, not yours. (You wouldn't share your email account or GitHub credentials with a colleague, right?) As of now, nobody has solved Simon's Lethal Trifecta - If a single agent has access to private information, can communicate externally, and is exposed to untrusted content, there is no structural way to guarantee that that agent can't be suborned. So the name of the game is compartmentalization and risk *reduction*.

The architecture I've landed on for now:

- The main agent does not have the ability to directly communicate externally.
- The main agent is able to communicate with ephemeral subagents that *do* have the ability to communicate externally

Modern self-help contains an in-built flaw:

To continually improve yourself, you must continually locate the ways you are broken.

Fortunately, there are a few perspective shifts that make all the difference. It took me embarrassingly long to figure them out.

To get started, let’s take a fresh look at an old concept.

MASLOW’S HIERARCHY OF NEEDS?

> **“I suppose it is tempting, if the only tool you have is a hammer, to treat everything as if it were a nail.”**
> — Abraham Maslow

Maslow’s Hierarchy of Needs has captured the minds of hundreds of millions. It offers simplicity in a terrifyingly complex world.

Abraham Maslow’s “A Theory of Human Motivation” (1943) contains five levels, which are typically presented like the below pyramid. This one is pulled from the Wikipedia entry on the subject:

We’ve all seen it. Clear as day, you can see the goal post at the top: self-actualization.

LFG! It’s time to journal and 80/20 myself! Pass me a shaman and some modafinil.

That’s the mission. That’s the point.

Right?

But hold on. A critical footnote got lost in the shuffle. In his later writings, especially notes compiled in *The Farther Reaches of Human Nature* (1971), Maslow added a *sixth* level above self-actualization:

Self-transcendence.

That update never quite made it out of the crib. The consultants are to blame, but that comes later.

Self-transcendence means going beyond the self — seeking connection with something greater, such as service to others, nature, art, or the divine. Why is it important? Well, for one thing, as Tony Robbins put it at an event long ago: "'I, I, I, me, me, me' gets to be a really fucking boring song."

But it's not just a boring song; it's dangerous to your health.

DON'T BE A SOMO

> **"The man who renounces himself, comes to himself."**
> — Ralph Waldo Emerson

Self-help is dangerous precisely because it easily becomes self-fixation.

A focus on improving the self usually first requires finding problems with the self. This is quite the pickle. In a society that rewards problem-solving, you can end up hallucinating or exaggerating unease in order to fix it. This leaves you always in the red, always one step behind. Imagine a dog chasing its tail that has committed to being unhappy until it catches the tail… but it's always just a few inches short. Still, it whirls around and around, "doing the work." Perfection always recedes by one more book, one more seminar, one more habit tracker.

Put in more colorful terms, misdirected self-help turns you into a self-obsessed masturbatory ouroboros (SOMO).

To remind me of the SOMO risk, I have this sticker on my laptop:

A picture is worth a thousand social media posts about yourself. Sticker from Porous Walker.

Now, to be clear, I still love self-help. Ain't no way Timmy can give up the sauce. There's a place for it.

From The Bible to Seneca, and from Ben Franklin to Stephen Covey and far beyond, there's a lot of valuable advice worth taking. I used to mainline it all – no time to waste! – and jump straight into action. This did some good, but there was a lot of collateral damage.

Why?

Because there are at least three "tectonic plates of self-help" that I couldn't see for decades, and they dictate how much net-positive or net-negative comes from all the striving. Before you sprint, you want to calibrate your direction.

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 08 JUL 2026 · SOURCE

You can also read this post on Prime Radiant's blog.

Almost all of this post was written on June 1, before Anthropic shipped Claude Tag. Nothing I'm writing about uses (or is based on) Claude Tag

At Prime Radiant, we've got a bunch of agents in our Slack. I want to tell you a bit about how we've started shipping some changes to internal production with an "agentic user in the loop." Letting an agent work directly with the implementer building its runloop and tooling is getting us higher-quality tools and experiences for the agent without a human in the middle of a game of telephone.

But first, a bit of backstory on how we're using non-coding agents at Prime Radiant.

One of them, Scribble, is just there to listen for when someone reports a problem or provides a new bit of information. It opens tickets for the issues and updates an internal wiki with the information. (It also logs our daily standups and keeps us honest about whether we finished what we said we were going to do the previous day.)

Another, Nora, is our junior go-to-market "person." Nora is built on top of Nanoclaw and is still learning her craft, but is surprisingly helpful, partially because she considers herself a team member, not just an assistant. I've put a bunch of time into tuning her prompting and constitution. She gets a ton of value out of knowing that she journals obsessively. (I should note that I don't tend to gender my agents, but Nora picked a name and a gender and was very clear about that and...who am I to argue?)

I think the most surprising interaction I've had with Nora was the morning I woke up to a slack DM telling me that I needed to be speaking at more conferences. (I do.) She pointed out that the CFP for a particular AI conference had closed a week earlier, but that the program chair was known to take late submissions of interesting talks by DM. She'd taken the liberty of drafting a DM for me to send. I didn't actually *send* the DM, but I should have.

We have another agent, "Spec-together" that was a first prototype of what has become our new multi-player Brainstorm app.

- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.

"How does this apply juniors? You can't expect them to really do all of that?"

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, … — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.

You know what *you* should own? A subscription to this newsletter:

THE THREE TECTONIC PLATES OF SELF-HELP

> **"As to methods there may be a million and then some, but principles are few. The man who grasps principles can successfully select his own methods. The man who tries methods, ignoring principles, is sure to have trouble."**
> — Harrington Emerson

In the last few years, my life has become much more of a joy than a grind, and that's because I've focused on three tectonic plates.

Let's take a close look at each.

1. Intention

Individual or Social?

Americans, in particular, worship at the altar of the rugged individualist. There are clear upsides to this. But steeped in a culture – offline and especially online – that puts the self on a pedestal, we can take self-improvement to be an end unto itself: a better self.

But is it an end unto itself? Does it automatically produce good things? I now have my doubts.

Here's one analogy I've drawn for myself.

Let's pretend that life is the game of soccer. You can work on the mechanics of soccer by yourself. You can always get better at dribbling, shooting, and running drills as a solo practitioner. You can read dozens of books, study tape, and earn a PhD in the physics of ball flight. You can post videos of stunning shots on YouTube and get showered by emojis.

But none of this is actually playing the game of soccer.

You can spend your whole life preparing for, instead of playing, the game of life.

But why would anyone, including yours truly, succumb to this?

Subconsciously, it spares you from the messiest but most rewarding game of all: human interaction. Perhaps people hurt or traumatized you long ago. You might also justify the endless polishing, as I did, with some version of "Once I've perfected myself, then I'll be ready for relationships." But here's the rub: that practice is exactly endless. You can always get better at dribbling and penalty kicks.

Digging further, focusing on improving the self is often in service of trying to control the world, especially if things were unpredictable or unstable when growing up. Banish emotion, live by spreadsheets, and all can be well. All can be *controlled*, or so the illusion goes. But as soon as you're interacting with – let alone depending on – other people, control as a construct goes out the window. And so we consciously or subconsciously avoid the messiness. This is also one of the reasons why a lot of optimizing achiever folks have a hard time in intimate relationships.

So how do I think about "self-help" now, having realized all of the above?

It is refreshingly simple: the goal is to build and improve my relationships. The sooner you get on the real field with real players, the sooner you can get to playing soccer and engaging with life. No more auto-fellating, even with the best of intentions. We've evolved over millions of years to be deeply social creatures, and the more you dodge that IN REAL PHYSICAL LIFE, the more you will suffer. This is why solitary confinement in prisons is often considered cruel and unusual punishment… and yet we do it to ourselves all the time.

There are a few questions that help corral this tectonic plate of *intention*:

- How does any given "self-help" help me in my relationships, and how can I apply it with other people *today* or *this week*?
- How can I take the ship out of the harbor and test it where it counts?

2. Audience

Do you have an audience for your self-development? If so, be careful.

Nary a minute can be spent on social media without bumping into a CAPS-rich "HOW X CHANGED MY LIFE" or a photo carousel of an ayahuasca retreat. If only Costa Rica got a dime for every bikini-clad healer under a waterfall!

Welcome to the theater of performative self-help. I won't belabor this, as we've all seen it, but I suggest reading about the insidious creep of audience capture here, and don't forge ahead in the fame game before reading 11 reasons not to become famous. It's hard to put the genie back in the bottle, so you should know what that genie will do to your life.

But the truth is that most of us aren't extreme examples of this. But even minor tendencies in this direction can do extreme damage over time.

- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?
- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?
- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven't confirmed yet?
- Think about how you'd test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?
- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!
- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say "here's what I built, under these constraints, with these tradeoffs?"
- Test it manually. Yes, there's automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?
- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?
- If you think your colleagues needs to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.
- Do customers need to know? Who reported the bug? Who's blocked? Let them know.
- Does the world need to know? Announce that it's out.

Ownership

THORSTEN BALL · 08 JUL 2026 · SOURCE

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.

Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase "ownership" or allude to it, but haven't explained what "ownership" means to me in a while.

So, ownership.

If I ask you "can you own this?" or "can you take care of this?" or "are you on it?" — what I'm doing is I'm asking you to *own* it, to own the solution of a problem from end to end. From "we have a problem" to "we don't have to think about it again."

That means, when you say that you're owning something, the expectation is that you…

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we're actually trying to solve here. Maybe you think "the problem is that we need to migrate from using X to using Y", but that's not a problem, that's a solution. The problem is likely something like "performance is bad", "it's not stable", "it fails for customer x". Maybe there's other possible solutions to that? Think about those. What are the tradeoffs? What's the best solution to go with considering these tradeoffs?

Below are a few questions that I've found helpful for nudging this particular tectonic plate in the right direction:

- If you couldn't tell a soul about "the work" you're doing, would you still do it? If not, you're not developing yourself; you're curating yourself.
- How has sharing your personal development created tradeoffs?
- If you *had to* take down 20% of your most popular posts, which would you take down and why?
- Are you describing strong catalysts (psychedelics, The Hoffman Process, you name it) instead of doing the post-session integration that makes them truly valuable?
- Have you become more robust or more fragile by offering your inner workings up to public vote?
- Has your social presence made you more or less of the person you want to be? How would the you of three or five years ago feel about your last year of posts? What about the you of 10 years from now?

3. Assumption

What are the fundamental assumptions behind your doing "the work"?

Let's begin with a Buddhist parable that I first heard from the incredible Jack Kornfield.

> *The old Master points to a big boulder and asks a disciple, "See that large rock over there?"*
>
> *"Yes", says the disciple.*
>
> *"Do you think it's heavy?" continues the Master.*
>
> *"Yes, it's very heavy!" replies the student.*
>
> *"Only if you pick it up," smiles the Master.*

Once again, the fundamental assumption behind self-help is often this: Something is not OK. Something is wrong. Something is not enough. Something needs fixing. If I can't find it, I'll create it.

We've established this. But there is a follow-on assumption that matters a lot.

If I fix the things that aren't OK, all will be well. If I improve myself enough, if I only work hard enough, I can finally eliminate my suffering.

I hate to inform you, but this doesn't work. I'm also thrilled to inform you that this doesn't work. You can stop picking up a lot of boulders.

There is one book that most opened my eyes to this reframe – *Already Free: Buddhism Meets Psychotherapy on the Path of Liberation* by Bruce Tift. It offers a terrifying but ultimately liberating realization: there is no perfect escape from suffering. It doesn't exist. But there is a way to find your long-sought unclenching, and it lies in cultivating your skill of acceptance as much as that of improvement.

Now, I can hear the chorus: *Has Tim gone soft? Given up the good fight? Is he telling everyone to chill after he himself red-lined and got the spoils? How convenient! And…*

Hold on a second. I'm telling you – intelligent acceptance is high-leverage. It's probably one of the highest forms of leverage. This is an approach that helps preserve your energy for where it really matters. My early forays into Stoicism and Seneca The Younger helped set the conditions for my biggest wins from 2004-2010. Still, I only learned a small fraction of what I needed.

So how do you cultivate your skill of acceptance without becoming complacent?

This is a big question and what I love about Bruce's book. Compared to a strictly Western or purely Eastern book, he blends them and offers a surgical guide to using *both* action and acceptance. You don't have to be a bull in a china shop or a cow in the rain; there is a middle path. That middle path is where all the gold is buried.

If the only tool you have is "self-improvement," you'll become a hammer looking for nails in a world that is 50% screws. I tried it. It can create the veneer of success, but it will leave your inner world in turmoil.

Suffice to say, the dual dance is the most joyful. Upgrade your toolkit with that in mind. Read Bruce's book. If it doesn't click, try *Radical Acceptance: Embracing Your Life with the Heart of a Buddha* by Tara Brach, which had a large impact on my life a decade before I found Bruce's book. In a sense, the writing of Seneca prepared me for Tara, which then prepared me for Bruce. So grab them all and thank me later.

If you want serenity, you need to be able to put the Serenity Prayer into practice. Seriously, I read it all the time.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there[3].

When AI people talk about LLMs having personalities, or wanting things, or even having souls[4], these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a recent paper about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

   ↩

2. You could explain this in terms of "the stories we tell ourselves". Many people (though not all) think that human identities are narratively constructed.

   ↩

3. I wrote about this last year in *Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality*. A little nudge to change Grok's views on South African internal politics can cause it to start calling itself "Mecha-Hitler".

   ↩

4. I have long believed that Claude "feels better" to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its "soul"). My guess is that if you tried to make a "less human" version of Claude, it would become rapidly less capable.

   ↩

> In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system**.

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training[1].

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess[2]. AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

MASLOW'S HAMBURGER OF NEEDS?

> **"The more one forgets himself—by giving himself to a cause to serve or another person to love—the more human he is and the more he actualizes himself. What is called 'self-actualization' is not an attainable aim at all, for the simple reason that the more one would strive for it, the more he would miss it. In other words, self-actualization is possible only as a side-effect of self-transcendence."**
> — Viktor E. Frankl

How can we easily keep ourselves on the right track?

As I remind myself these days: **It's the relationships, stupid.**

For a nice simple visual, let's revise Maslow's pyramid with all of this in mind. This is easy, as Maslow never drew his model as a rigid pyramid!

He described "classes" of needs that were unfixed, overlapping, and that could reverse in order. And believe it or not, self-actualization was only ever for the "self-actualizing minority." In the 1960s, his work was co-opted by consultants and corporate trainers who needed a progression to sell. True story.

Given all this, and after decades of trial and error, here's where I've landed:

Maslow's Hamburger of Needs.

Ahhh… what? Not to worry. It's the same good ol' Maslow ingredients, but I think of it as a hamburger:

For our purposes, the meat, the whole point of the hamburger, is that middle layer: relationships. That is the center of life. The heartbeat.

As luck would have it, when you improve the heartbeat, it also feeds everything else.

You'll notice that the meat contains Abe's most-important addendum—the sixth level of self-transcendence. Focusing on things bigger than yourself is a critical piece of the ultimate puzzle. Faith, nature, family, meditation, causes that outlive you, etc. – take your pick. But be careful. If you do it to inflate the ego or impress others, it's self-obsession again, not self-transcendence. If you need credit, it doesn't count.

Of course, it should go without saying but the top and bottom layers matter a lot. A hamburger is a giant mess without the bun. Friends will get sick of you crashing on their couch and eating their food.

But the bread and dressing layers exist to serve the middle. That's the payload. Everything is in service of the payload. And the payload circulates benefits back to the edges, and then the cycle repeats. Even if you think this is oversimplified claptrap, temporarily assuming it's true will help you.

What if nearly everything you focused on — calendar, habits, goals — aimed to improve your relational life somehow? What if you took this as a challenge for even a week? Your lens on the world changes dramatically.

You say yes differently.
You say no more clearly.
Your to-do list for life slowly transforms.

What if all that you focused on, all that you do, had to improve that middle layer in some fashion?

It's a damn hard question if you've been on the *self*-help train for a while. I get it.

So let's try something easier: **What if it only changed how you approach your to-do list? Try hamburger-first each day for 1-2 weeks and tell me what happens. Add and do the things that improve your relational life FIRST. Nothing on the list? Create something. It could be as simple as cooking dinner for your spouse, complimenting at least three people a day for a week, or introducing yourself to the barista you see every morning. Getting started is how you get grooving.**

ARE YOU DOING SELF-HELP, OR IS SELF-HELP DOING YOU?

> **For friendship makes prosperity more shining and lessens adversity by dividing and sharing it.**
> — Marcus Tullius Cicero

In his *Moral Letters to Lucilius*, Seneca the Younger famously wrote that "These individuals [who put money at the center of life] have riches just as we say that we 'have a fever,' when really the fever has us."

What if self-help is similar?

Obsessing over the self never provides peace. It cannot make you whole, as you aren't the whole. Becoming whole starts by putting down the rock you didn't even know you were carrying.

Because at the end of the day—and at the end of a Montana night—the point was never yourself.

It was never the pyramid.

It was never the optimization.

It was the people around the fire.

Share this:

- Share on X (Opens in new window) X
- Share on Facebook (Opens in new window) Facebook
- Share on LinkedIn (Opens in new window) LinkedIn
- Email a link to a friend (Opens in new window) Email
- More

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · SOURCE

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this opinion piece by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into AI psychosis. Here's a representative paragraph from the piece: